

Lab Assignment # 10

Course Title : **AI Assistant Coding**
Name of Student : **R Govardhan Sai Ganesh**
Enrollment No. : **2303A54044**
Batch No. : **48**

Lab 10: Code Review and Quality: Using AI to improve code quality and readability

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```

Instructions:

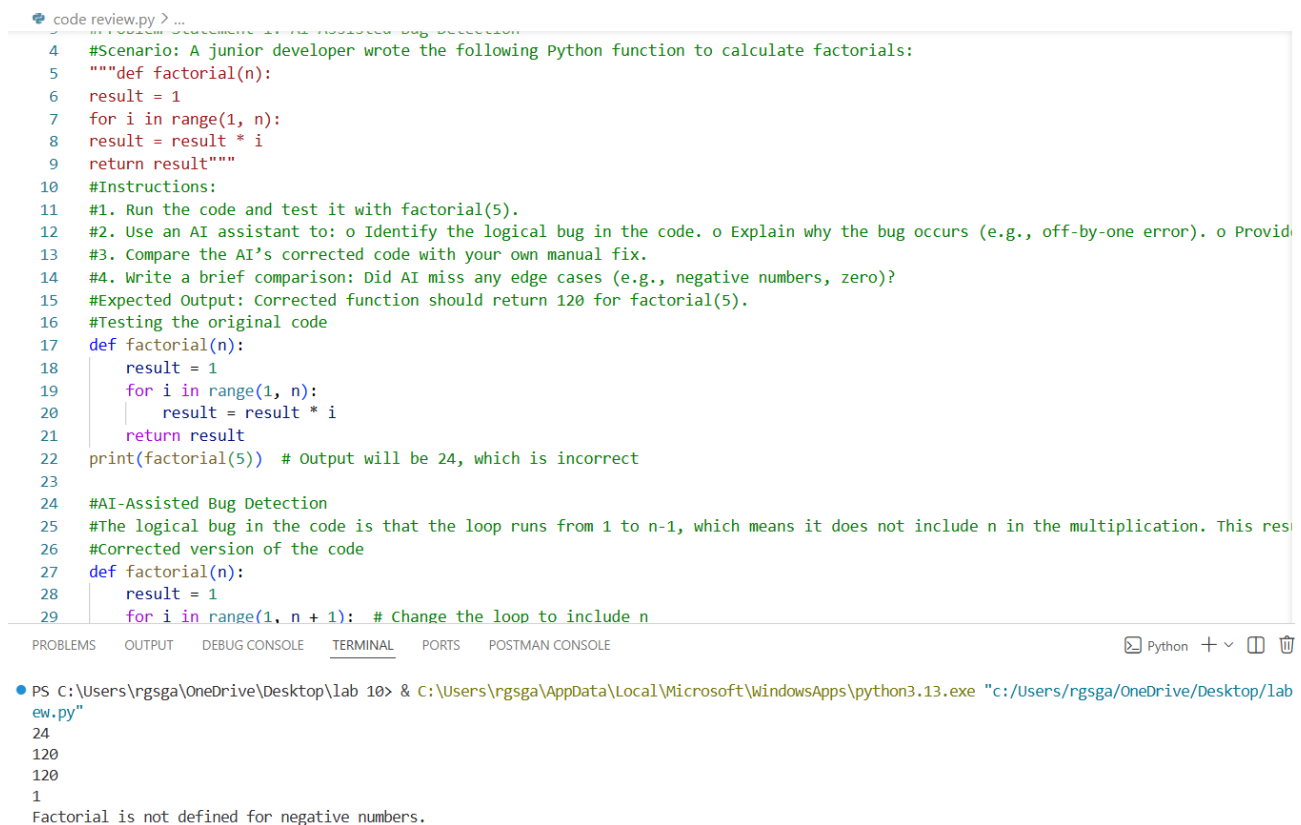
1. Run the code and test it with factorial (5).
2. Use an AI assistant to:
 - o Identify the logical bug in the code.
 - o Explain why the bug occurs (e.g., off-by-one error).
 - o Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Expected Output: Corrected function should return 120 for factorial (5).

Brief Comparison: The AI correctly identified the off-by-one error and provided a corrected version of the code. However, both the original and corrected versions do not handle edge cases such as negative numbers or zero. The factorial function should ideally return 1 for factorial(0) and raise an error for negative inputs. This is an area where the AI could have provided additional improvements to enhance the robustness of the code.

The logical bug in the code is that the loop runs from 1 to n-1, which means it does not include n in the multiplication. This results in an off-by-one error, causing the function to return 24 instead of 120 for factorial(5).

Output Screenshot:



The screenshot shows a code editor with a Python file named 'code review.py'. The code defines a factorial function and includes instructions for testing and AI-assisted bug detection. The terminal output shows the function being called with 5, resulting in 120, and a message indicating that the original code was incorrect for negative numbers.

```
code review.py > ...
4 #Scenario: A junior developer wrote the following Python function to calculate factorials:
5 """def factorial(n):
6     result = 1
7     for i in range(1, n):
8         result = result * i
9     return result"""
10 #Instructions:
11 #1. Run the code and test it with factorial(5).
12 #2. Use an AI assistant to: o Identify the logical bug in the code. o Explain why the bug occurs (e.g., off-by-one error). o Provide
13 #3. Compare the AI's corrected code with your own manual fix.
14 #4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?
15 #Expected Output: Corrected function should return 120 for factorial(5).
16 #Testing the original code
17 def factorial(n):
18     result = 1
19     for i in range(1, n):
20         result = result * i
21     return result
22 print(factorial(5)) # Output will be 24, which is incorrect
23
24 #AI-Assisted Bug Detection
25 #The logical bug in the code is that the loop runs from 1 to n-1, which means it does not include n in the multiplication. This results
26 #Corrected version of the code
27 def factorial(n):
28     result = 1
29     for i in range(1, n + 1): # Change the loop to include n
30
31 PS C:\Users\rgsga\OneDrive\Desktop\lab 10> & C:\Users\rgsga\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:/Users/rgsga/OneDrive/Desktop/lab
32 ew.py"
33 24
34 120
35 120
36 1
37 Factorial is not defined for negative numbers.
```

Problem Statement 2: Task 2 — Improving Readability & Documentation

Scenario: The following code works but is poorly written:

```
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
```

Instructions:

Use AI to:

- o Critique the function's readability, parameter naming, and lack of documentation.
- o Rewrite the function with:
 1. Descriptive function and parameter names.
 2. A complete docstring (description, parameters, return value, examples).
 3. Exception handling for division by zero.
 4. Consideration of input validation.
 6. Compare the original and AI-improved versions.
 7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

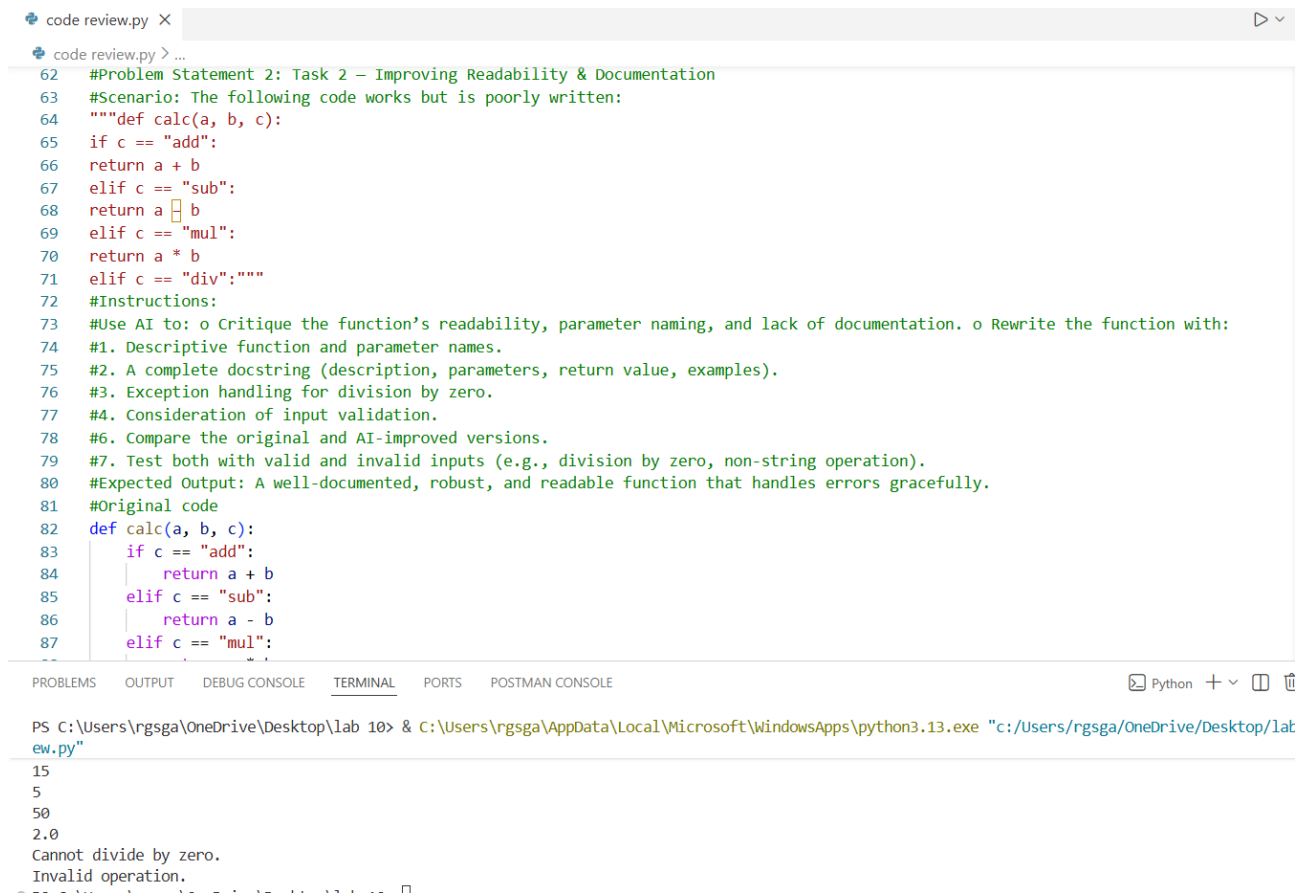
Expected Output: A well-documented, robust, and readable function that handles errors gracefully.

Brief Comparison: The original code is functional but lacks readability, proper naming conventions, and documentation. The AI-improved version addresses these issues by providing descriptive names, a comprehensive docstring, and error handling for invalid operations and division by zero. The improved

version is more robust and user-friendly, making it easier for other developers to understand and maintain the code.

Overall, the AI-assisted improvements significantly enhance the quality of the code, making it more maintainable and less prone to errors.

Output Screenshot:



The screenshot shows a code editor with a file named 'code review.py'. The code defines a function 'calc(a, b, c)' that performs arithmetic operations based on the value of 'c'. The function is poorly documented and has some syntax errors. The code is as follows:

```
62 #Problem Statement 2: Task 2 – Improving Readability & Documentation
63 #Scenario: The following code works but is poorly written:
64 """def calc(a, b, c):
65     if c == "add":
66         return a + b
67     elif c == "sub":
68         return a - b
69     elif c == "mul":
70         return a * b
71     elif c == "div":
72         return a / b
73 #Instructions:
74 #Use AI to:
75 #1. Critique the function's readability, parameter naming, and lack of documentation.
76 #2. Rewrite the function with:
77 #3. Descriptive function and parameter names.
78 #4. A complete docstring (description, parameters, return value, examples).
79 #5. Exception handling for division by zero.
80 #6. Consideration of input validation.
81 #6. Compare the original and AI-improved versions.
82 #7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).
83 #Expected Output: A well-documented, robust, and readable function that handles errors gracefully.
84 #Original code
85 def calc(a, b, c):
86     if c == "add":
87         return a + b
88     elif c == "sub":
89         return a - b
90     elif c == "mul":
91         return a * b
92     elif c == "div":
93         return a / b
```

The terminal output shows the execution of the function with various inputs:

```
PS C:\Users\rgsga\OneDrive\Desktop\lab 10> & C:\Users\rgsga\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:/Users/rgsga/OneDrive/Desktop/lab 10/code review.py"
15
5
50
2.0
Cannot divide by zero.
Invalid operation.
```

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

Instructions:

- Verify the function works correctly for sample inputs.
- Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:
 - List all PEP8 violations.
 - Refactor the code (function name, spacing, indentation, naming).
- Apply the AI-suggested changes and verify functionality is preserved.
- Write a short note on how automated AI reviews could streamline code reviews in large teams.

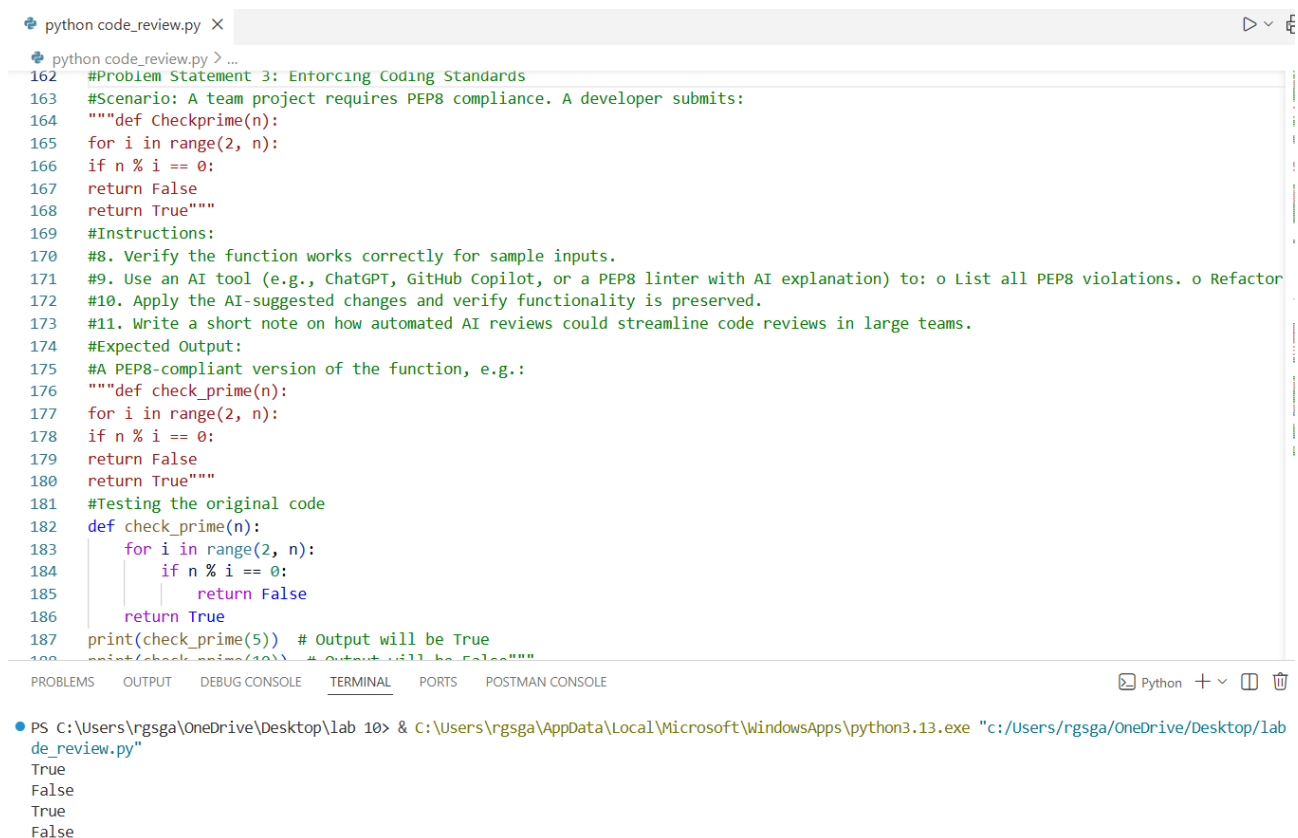
Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def check_prime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

Short Note: Automated AI reviews can significantly streamline code reviews in large teams by quickly identifying coding standard violations, suggesting improvements, and providing explanations for changes. This can save time for human reviewers, allowing them to focus on more complex issues such as architecture and design. Additionally, AI can help maintain consistency across the codebase, especially in teams with varying levels of experience. However, it's important to use AI as a tool to assist human judgment rather than replace it entirely, as there may be context-specific considerations that AI might not fully understand.

Output Screenshot:



```
python code_review.py X
python code_review.py > ...
162 #Problem Statement 3: Enforcing Coding Standards
163 #Scenario: A team project requires PEP8 compliance. A developer submits:
164 """def checkprime(n):
165     for i in range(2, n):
166         if n % i == 0:
167             return False
168         return True"""
169 #Instructions:
170 #8. Verify the function works correctly for sample inputs.
171 #9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to: o List all PEP8 violations. o Refactor
172 #10. Apply the AI-suggested changes and verify functionality is preserved.
173 #11. Write a short note on how automated AI reviews could streamline code reviews in large teams.
174 #Expected Output:
175 #A PEP8-compliant version of the function, e.g.:
176 """def check_prime(n):
177     for i in range(2, n):
178         if n % i == 0:
179             return False
180         return True"""
181 #Testing the original code
182 def check_prime(n):
183     for i in range(2, n):
184         if n % i == 0:
185             return False
186         return True
187 print(check_prime(5)) # Output will be True
188 print(check_prime(4)) # Output will be False"""
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
Python + - [ ] [X]
PS C:\Users\rgsga\OneDrive\Desktop\lab 10> & C:\Users\rgsga\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:/Users/rgsga/OneDrive/Desktop/lab
de_review.py"
True
False
True
False
```

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario: In a GitHub project, a teammate submits:

```
def processData(d):
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:
 - o Readability and naming.
 - o Reusability and modularity.
 - o Edge cases (non-list input, empty list, non-integer elements).
2. Use AI to generate a code review covering:
 - a. Better naming and function purpose clarity.
 - b. Input validation and type hints.

- c. Suggestions for generalization (e.g., configurable multiplier).
3. Refactor the function based on AI feedback.
4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

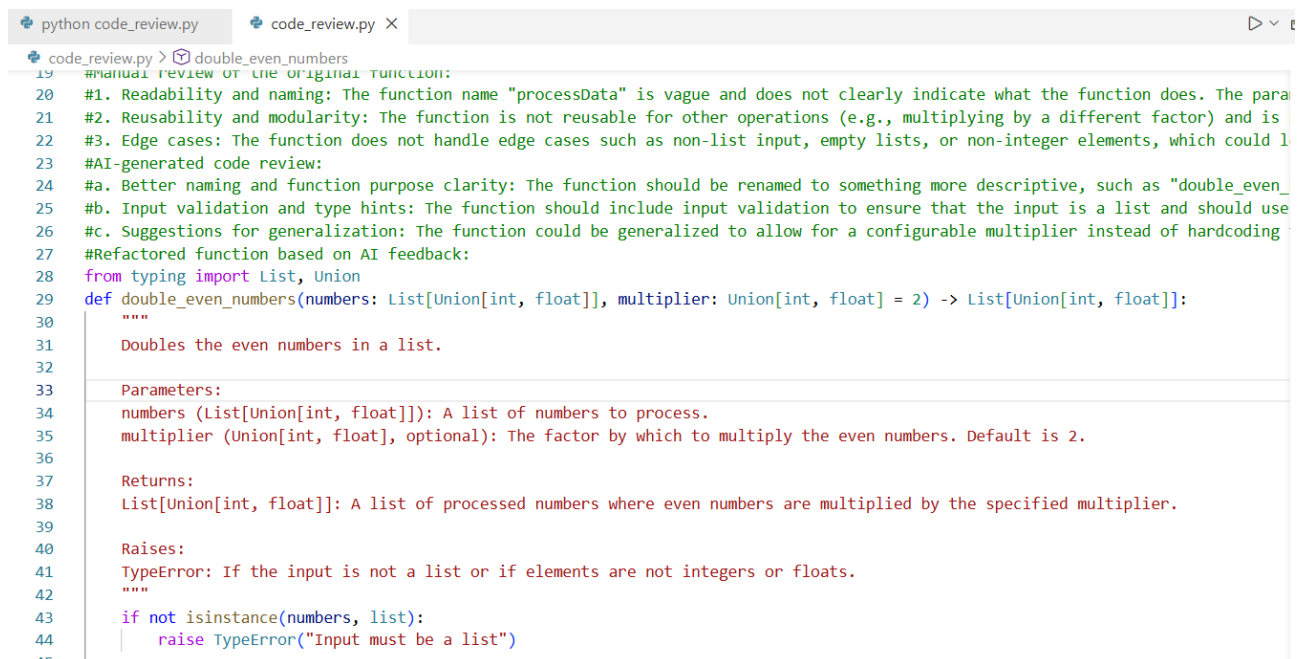
Expected Output: An improved function with type hints, validation, and clearer intent, e.g.:

```
from typing import List, Union
def double_even_numbers(numbers: List[Union[int, float]]) -> List[Union[int, float]]:
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")
    return [num * 2 for num in numbers if isinstance(num,
(int, float)) and num % 2 == 0]
```

Reflection on AI as a standalone reviewer vs. an assistant: AI can be a valuable tool in code reviews, especially for identifying common issues such as naming conventions, input validation, and code generalization. However, AI should be used as an assistant rather than a standalone reviewer. Human judgment is crucial for understanding the context of the code, making decisions about design and architecture, and considering edge cases that may not be immediately apparent to AI. Additionally, AI may not always understand the nuances of a project or the specific requirements of a task, which is why human oversight is essential to ensure that the code meets the project's standards and goals.

In conclusion, while AI can enhance the code review process by providing suggestions and identifying potential issues, it should complement human reviewers rather than replace them. The combination of AI's efficiency and human insight can lead to higher-quality code and more effective reviews.

Output Screenshot:



```
python code_review.py  code_review.py X
code_review.py > double_even_numbers
19 #manual review of the original function:
20 #1. Readability and naming: The function name "processData" is vague and does not clearly indicate what the function does. The para
21 #2. Reusability and modularity: The function is not reusable for other operations (e.g., multiplying by a different factor) and is
22 #3. Edge cases: The function does not handle edge cases such as non-list input, empty lists, or non-integer elements, which could l
23 #AI-generated code review:
24 #a. Better naming and function purpose clarity: The function should be renamed to something more descriptive, such as "double_even_
25 #b. Input validation and type hints: The function should include input validation to ensure that the input is a list and should use
26 #c. Suggestions for generalization: The function could be generalized to allow for a configurable multiplier instead of hardcoding
27 #Refactored function based on AI feedback:
28 from typing import List, Union
29 def double_even_numbers(numbers: List[Union[int, float]], multiplier: Union[int, float] = 2) -> List[Union[int, float]]:
30     """
31     Doubles the even numbers in a list.
32
33     Parameters:
34     numbers (List[Union[int, float]]): A list of numbers to process.
35     multiplier (Union[int, float], optional): The factor by which to multiply the even numbers. Default is 2.
36
37     Returns:
38     List[Union[int, float]]: A list of processed numbers where even numbers are multiplied by the specified multiplier.
39
40     Raises:
41     TypeError: If the input is not a list or if elements are not integers or floats.
42     """
43     if not isinstance(numbers, list):
44         raise TypeError("Input must be a list")
45
```

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
    return total
```

Instructions:

1. Test the function with a large list (e.g., `range(1000000)`).
2. Use AI to:
 - o Analyze time complexity.
 - o Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
 - o Provide an optimized version.
3. Compare execution time before and after optimization.
4. Discuss trade-offs between readability and performance.

Expected Output:

An optimized function, such as:

```
def sum_of_squares_optimized(numbers):
    return sum(x * x for x in numbers)
```

Comparing execution times: The execution time for the original function will likely be significantly longer than the optimized version, especially as the size of the input list increases. The optimized version should show a noticeable improvement in performance, demonstrating the benefits of using built-in functions and generator expressions in Python.

Output Screenshot:

